

I wouldn't ignore it completely, but it's not an immediate problem.

The warning means that somewhere in your code (or in a library you're using), a URL is being passed like:

Python

```
www.swr3.de/article123
```

instead of:

Python

```
https://www.swr3.de/article123
```

Current versions of `urllib3` try to be forgiving, but a future version will raise an exception instead.

First, identify which URL is triggering it

Add some debugging before calling `trafilatura.fetch_url()`:

Python

```
url = story["url"]  
print(repr(url))
```

Look for entries that don't start with `http://` or `https://`.

You can also check your JSON:

Python

```
for story in stories:  
    url = story["url"]  
  
    if not url.startswith(("http://", "https://")):  
        print("BAD:", url)
```

Common RSS issue

Some feeds contain relative links:

```
/news/article123.html
```

or

```
www.swr3.de/news/article123.html
```

In that case normalize them:

Python

```
from urllib.parse import urljoin

BASE_URL = "https://www.swr3.de"

url = story["url"]

if not url.startswith(("http://", "https://")):
    url = urljoin(BASE_URL, url)
```

If all URLs look correct

Then the warning may be coming from inside `trafilatura` when it processes images, feeds, redirects, or metadata.

You can quickly test:

Python

```
import requests

for story in stories:
    requests.get(story["url"], timeout=10)
```

If that runs without warnings, the issue is likely internal to `trafilatura` and not your code.

My guess

Since you're working from an RSS feed, the most likely culprit is one or two entries whose `link` field is missing the scheme. I'd inspect the first few URLs with:

Python

```
for story in stories[:20]:
    print(story["url"])
```

If you paste one or two sample entries from your `stories.json`, I can tell immediately whether the feed data needs normalization or whether the warning is coming from a dependency.



I see entries with an empty URL in `stories.json`. How can I handle that?

That's very likely the cause.

An empty URL (`""`) gets passed down to `requests / urllib3`, which triggers the "URL without a scheme" warning because it effectively tries to process a non-URL.

Filter them out early

When building `stories.json`:

Python

```
for entry in feed.entries:
    url = entry.get("link", "").strip()

    if not url:
        continue

    stories.append({
        "title": entry.get("title", ""),
        "url": url,
        "published": entry.get("published", ""),
        "summary": entry.get("summary", ""),
    })
```

Or filter before downloading articles

If you've already generated `stories.json`:

Python

```
for story in stories:
    url = story.get("url", "").strip()

    if not url:
        print(f"Skipping empty URL: {story.get('title')}")
        continue

    downloaded = trafilatura.fetch_url(url)
```

Add stronger validation

I'd recommend validating URLs explicitly:

Python

```
from urllib.parse import urlparse

def is_valid_url(url):
    try:
        parsed = urlparse(url)
        return parsed.scheme in ("http", "https") and parsed.netloc
    except Exception:
        return False
```

Usage:

Python

```
url = story.get("url", "").strip()

if not is_valid_url(url):
    print(f"Skipping invalid URL: {url!r}")
    continue
```

Investigate why they're empty

For SWR3, I'd also inspect the raw feed data:

Python

```
for entry in feed.entries:
    if not entry.get("link"):
        print(entry)
```

Sometimes RSS feeds contain items that are:

- podcast enclosures,
- image-only entries,
- malformed feed items,
- promotional content,

and those can legitimately lack a `link` .

For a news podcast pipeline, it's perfectly fine to skip any item without a valid article URL. In fact, I'd generally discard any item that fails one of these checks:

Python

```
if (
    not url
    or not is_valid_url(url)
    or len(story.get("title", "")) < 10
):
    continue
```

That will make the downstream article extraction stage much more reliable.



Could you get me an updated `fetch_swr3.py` ?

Yes. I'd update the script to:

- skip empty URLs,
- validate URLs before saving,
- normalize whitespace,
- emit some stats so you can see what's happening,
- write clean JSON for the next stage.

Python

```
#!/usr/bin/env python3

import json
from urllib.parse import urlparse

import feedparser

FEED_URL = "https://www.swr3.de/~rss/index.xml"

def is_valid_url(url: str) -> bool:
    if not url:
        return False

    try:
        parsed = urlparse(url)
        return (
            parsed.scheme in ("http", "https")
            and bool(parsed.netloc)
        )
    except Exception:
        return False

feed = feedparser.parse(FEED_URL)

print(f"Feed: {feed.feed.get('title', 'Unknown')}")
print(f"Entries in feed: {len(feed.entries)}")
print()

stories = []
skipped = 0

for entry in feed.entries:
    title = entry.get("title", "").strip()
    url = entry.get("link", "").strip()
    published = entry.get("published", "").strip()
    summary = entry.get("summary", "").strip()

    if not is_valid_url(url):
        skipped += 1

        print(
            f"Skipping invalid URL "
            f"(title={title!r}, url={url!r})"
        )

        continue
```

```
        stories.append(
            {
                "title": title,
                "url": url,
                "published": published,
                "summary": summary,
            }
        )

    print()
    print(f"Valid stories: {len(stories)}")
    print(f"Skipped: {skipped}")

    with open("stories.json", "w", encoding="utf-8") as f:
        json.dump(
            stories,
            f,
            ensure_ascii=False,
            indent=2,
        )

    print("\nWrote stories.json")
```

A further improvement I'd recommend is to save the raw problematic entries for inspection:

Python

```
bad_entries = []

...

if not is_valid_url(url):
    bad_entries.append({
        "title": title,
        "url": url,
        "raw": dict(entry),
    })
    continue
```

and then:

Python


```
with open("bad_entries.json", "w", encoding="utf-8") as f:
    json.dump(bad_entries, f, ensure_ascii=False, indent=2)
```

That way you can determine whether SWR3 is actually publishing malformed items or whether `feedparser` is interpreting some feed elements unexpectedly. The SWR3 RSS feed is published at <https://www.swr3.de/~rss/index.xml>.

 [rss-verzeichnis.de](https://www.swr3.de/~rss/index.xml) +1

One other thing: before moving on to article extraction, print the first few URLs:

Python

```
for story in stories[:5]:
    print(story["url"])
```

If every URL starts with <https://www.swr3.de/>, you've probably eliminated the warning entirely.



Sources

Can I set a custom OpenAI-compatible endpoint with Python's OpenAI? I use a custom provder.

Yes. The official OpenAI Python SDK supports OpenAI-compatible providers via the `base_url` parameter.

Example:

Python

```
from openai import OpenAI

client = OpenAI(
    api_key="your-api-key",
    base_url="https://your-provider.example.com/v1",
)
```

Then use it normally:

Python

```
response = client.responses.create(
    model="gpt-4.1",
    input="Hello",
)

print(response.output_text)
```

Or for chat completions (many OpenAI-compatible providers still expose this API):

Python

```
response = client.chat.completions.create(
    model="your-model",
    messages=[
        {"role": "user", "content": "Summarize this article"}
    ]
)

print(response.choices[0].message.content)
```

Environment variables

A nice pattern is:

Python

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["OPENAI_API_KEY"],
    base_url=os.environ.get("OPENAI_BASE_URL"),
)
```

Then:

Bash

```
export OPENAI_API_KEY=...
export OPENAI_BASE_URL=https://your-provider.example.com/v1
```

Common compatibility issues

Different providers vary in how closely they match OpenAI's API:

Feature	Usually works
Chat Completions	✓
Responses API	Often
Embeddings	Usually
Audio/TTS	Sometimes
Images	Varies
Streaming	Usually

If you get errors such as:

```
404 Not Found
```

or

```
Unknown endpoint /responses
```

try switching from:

Python

```
client.responses.create(...)
```

to:

Python

```
client.chat.completions.create(...)
```

since chat completions are the most widely implemented endpoint among OpenAI-compatible providers.

For your podcast project

I'd structure it like:

Python

```
client = OpenAI(  
    api_key=os.environ["LLM_API_KEY"],  
    base_url=os.environ["LLM_BASE_URL"],  
)
```

and keep the rest of the code provider-agnostic. Then you can switch between OpenAI, local servers (e.g. vLLM), or other compatible providers just by changing environment variables.

